

Deployment architectures

Mod_security can be deployed in two modes:

- Embedded mode: Just add *mod_security* as a module into your Apache Web server. However, in this mode, it is not able to inspect the content of server headers.
- Network gateway: In this mode (recommended since all the Web traffic goes through the proxy), *mod_security* is installed as a reverse proxy (see Figure 1). If you are using this mode, ensure you also add *mod_proxy* and *mod_proxy_http* in Apache. This gives a single point to monitor, higher speed, higher anonymity of the internal network, and *mod_security* can inspect the server header of the back-end database.

Installation

Installation is not straightforward, and also depends on the OS and Apache version you're using. But the slight trouble is negligible in comparison to the services *mod_security* offers. For detailed installation steps, refer to <http://bit.ly/p9izMG>. A Windows version is available at <http://www.apacheounge.com/download/> which also has other customised versions of Apache and Apache modules.

Configuration

Basically, *mod_security* works on configuration and rules. The configuration instructs it how to process the data it sees; the rules decide what to do with the processed data.

The configuration directives can be directly added to *httpd.conf*, but to avoid cluttering that, include a separate *modsecurity.conf* with this line in *httpd.conf*:

```
Include conf/modsecurity.conf
```

The basic *modsecurity.conf* looks like the following code:

```
<IfModule mod_security.>
# Turn the filtering engine On or Off
SecFilterEngine On
# The audit engine works independently and can be turned
# On or Off on a per-server or per-directory basis
SecAuditEngine RelevantOnly
# Make sure that URL encoding is valid
SecFilterCheckURLEncoding On
# Unicode encoding check
SecFilterCheckUnicodeEncoding On
# Only allow bytes from this range
SecFilterForceByteRange 1 255
# Cookie format checks.
SecFilterCheckCookieFormat On
# The name of the audit log file
SecAuditLog logs/audit_log
# Should mod_security inspect POST payloads
SecFilterScanPOST On
# Default action set
```

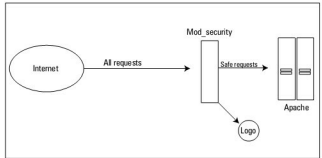


Figure 1: *mod_security* as reverse proxy

```
SecFilterDefaultAction "deny,log,status:500"
</IfModule>
```

Now, let's look at some basic configuration directives:

- *SecFilterEngine*: When set to On (that is, *SecFilterEngine* *On*), it starts monitoring requests. It is *Off* (disabled) by default.
- *SecFilterScanPOST*: When *On*, enables scanning the request body/POST payload.
- *SecFilterScanOutput*: When *On*, enables scanning the response body also.

Similarly, to check URL encoding, you can use *SecFilterCheckURLEncoding*; to control request body buffering, use *SecRequestBodyAccess*; to control what happens once the response body limit is reached, use *SecResponseBodyLimitAction*; and to specify the response body buffering limit, use *SecResponseBodyLimit*. The full list of configuration directives, their usage and syntax is at <http://bit.ly/oSAX5s>

Rules—the basics

The *mod_security* rule engine is where gathered data is checked for any malicious or particular content. Rules are directives in the configuration file that decide what to do with the data parsed by the configuration directives. The rule language is a vast topic; we'll only discuss basic rule-writing syntax, and rule directives to secure Web applications from all the attacks we've discussed so far.

The main directive used to create rules is *SecRule*, whose syntax is as follows:

```
SecRule VARIABLES OPERATOR [ ACTIONS]
```

- **VARIABLES**: Specify which places to check in an HTTP transaction. *Mod_security* pre-processes raw transaction data, making it easy for rules to focus on the logic of detection. Currently, variables are divided into *request*, *server*, and *response* variables, *parsing flags* and *time* variables. You can use multiple variables in a single rule with the *()* operator.
- **OPERATORS**: Specify a regular expression, pattern or keyword to be checked in the variable(s). There are four types of operators: *string-matching*, *numerical*,